Module 8: Multi-Party Computation (MPC)

Threshold Cryptography for Secure Signing

What is MPC?

Multi-Party Computation

- **Distributed Cryptography**: Multiple parties compute together without revealing secrets
- **Threshold Security**: t-of-n scheme (need t shares out of n total to sign)
- **Secure Signing**: Sign transactions without exposing private keys

Use Cases

- **Multi-sig Wallets**: Distributed control over funds
- **Secure Custody**: Institutional-grade key management
- **Decentralized Signing**: No single point of failure

Threshold Schemes

Common Configurations

- **2-of-3**: 2 shares needed, 3 total shares
- **3-of-5**: 3 shares needed, 5 total shares
- **4-of-7**: 4 shares needed, 7 total shares

Security Properties

- **t shares**: Minimum required for signing
- **n shares**: Total shares distributed
- **No single point**: No individual can sign alone
- **Flexible recovery**: Replace compromised shares

Architecture Overview

```
┌─────────────────────────────────────────────────────────────┐
│                         API Layer                           │
│  ┌──────────────────────────────────────────────────────┐   │
│  │              MpcController                           │   │
│  │ • POST /mpc/wallets → createMpcWallet()              │   │
│  │ • GET /mpc/wallets → getMpcWallets()                 │   │
│  │ • GET /mpc/wallets/:walletId → getMpcWallet()        │   │
│  │ • GET /mpc/wallets/:walletId/shares → getShares()    │   │
│  │ • POST /mpc/sign → signTransaction()                 │   │
│  │ • DELETE /mpc/wallets/... → revokeKeyShare()         │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
                      │
                      ▼
┌─────────────────────────────────────────────────────────────┐
│                       Service Layer                         │
│  ┌──────────────────────────────────────────────────────┐   │
│  │               MpcService                             │   │
│  │ • createMpcWallet() - Wallet creation                │   │
│  │ • getMpcWallets() - Wallet listing                   │   │
│  │ • getMpcWallet() - Wallet retrieval                  │   │
│  │ • getWalletKeyShares() - Share distribution          │   │
│  │ • signTransaction() - Threshold signing              │   │
│  │ • revokeKeyShare() - Share revocation                │   │
│  │ • generateDistributedKey() - DKG simulation          │   │
│  │ • reconstructSignature() - Signature reconstruction  │   │
│  │ • validateParticipantShares() - Share validation     │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
                      │
                      ▼
┌─────────────────────────────────────────────────────────────┐
│                        Database                             │
│  ┌──────────────────────────────────────────────────────┐   │
│  │            PostgreSQL + TypeORM                      │   │
│  │ • Repository<MpcWallet>                              │   │
│  │ • Repository<KeyShare>                               │   │
│  │ • One-to-Many: Wallet → Shares                       │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│                    Threshold Schemes                        │
│  ┌──────────────────────────────────────────────────────┐   │
│  │            Implemented Schemes                       │   │
│  │ • 2-of-3: 2 shares needed, 3 total                   │   │
│  │ • 3-of-5: 3 shares needed, 5 total                   │   │
│  │ • 4-of-7: 4 shares needed, 7 total                   │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
```

Threshold Signing

```
// Sign transaction with 2 shares (for 2-of-3 wallet)
const signature = await mpcService.signTransaction({
  walletId: wallet.id,
  transaction: serializedTransaction,
  shares: [
    { participantId: "alice", shareIndex: 0, encryptedShare: aliceShare },
    { participantId: "bob", shareIndex: 1, encryptedShare: bobShare }
  ]
});

// Use signature in Solana transaction
const tx = Transaction.from(Buffer.from(serializedTransaction));
tx.addSignature(wallet.publicKey, signature);
```

Error Handling

Common Errors

- **BadRequestException**: Invalid parameters or insufficient shares
- **NotFoundException**: Wallet or share not found
- **UnauthorizedException**: Invalid participant authentication
- **ForbiddenException**: Threshold not met for signing

Validation Checks

- Participant public key verification
- Share encryption validation
- Threshold requirement enforcement
- Wallet status verification

Use Cases & Benefits

Institutional Custody

- **Secure Key Management**: No single employee can access funds
- **Regulatory Compliance**: Audit trails and access controls
- **Business Continuity**: Share recovery for lost access

Decentralized Applications

- **DAO Treasury**: Multi-sig control for community funds
- **DeFi Protocols**: Secure oracle or bridge operations
- **NFT Marketplaces**: Escrow and royalty management

Performance Considerations

Optimization Strategies

- **Share Caching**: Cache decrypted shares for session
- **Batch Signing**: Process multiple transactions together
- **Connection Pooling**: Efficient database connections
- **Async Processing**: Non-blocking cryptographic operations

Scalability

- **Horizontal Scaling**: Multiple MPC service instances
- **Load Balancing**: Distribute signing requests
- **Rate Limiting**: Prevent abuse and DoS attacks

Security Best Practices

Operational Security

- **Secure Share Storage**: Encrypted cold storage for shares
- **Access Logging**: Complete audit trail of all operations
- **Regular Rotation**: Periodic key share rotation
- **Backup Procedures**: Secure backup and recovery processes

Cryptographic Security

- **Key Share Encryption**: Strong encryption for stored shares
- **Secure Communication**: TLS for all API communications
- **Participant Verification**: Multi-factor authentication
- **Compromise Response**: Immediate share revocation procedures

Integration with Solana

Transaction Signing

- **Ed25519 Compatibility**: Works with Solana's signature scheme
- **Transaction Serialization**: Standard Solana transaction format
- **Fee Management**: Integrated with fee estimation services

Wallet Abstraction

- **Unified Interface**: Same API as regular wallets
- **Transparent Operation**: Applications don't need MPC awareness
- **Fallback Support**: Regular keypair signing as backup

Next Steps

Module 8 Implementation Tasks

- ✅ Threshold signature generation
- ✅ Distributed key generation
- ✅ Key share management
- ✅ Signature reconstruction
- ✅ MPC wallet creation
- ✅ Secure key distribution
- ✅ MPC transaction signing
- ✅ Recovery mechanisms

Related Modules

- **Module 7**: Signing & Cryptography

Thank You!

Questions?

MPC enables secure, distributed control over blockchain assets!

Back to Study Topics ⬅️